

UNIVERSIDAD
DE GRANADA

Práctica 2

Problema del portfolio

Pablo Linari Pérez

DNI: 75571079W
Grupo de prácticas: 3 (Martes 15:30-17:30)
Curso: 25/26
Correo: pablolinari@correo.ugr.es

Índice de contenidos

1 Descripción del problema 3
2 Algoritmos empleados y estructura del problema 4
2.1 Estructura del problema 4
2.2 Representación de las soluciones 4
2.3 Funciones comunes usadas por los algoritmos 5
2.4 Operadores 7
2.4.1 Operadores de cruce 7
2.4.2 Operador de selección 9
2.4.3 Operador de mutación 9
3 Estructura de los algoritmos 11
3.1 Local Search 11
3.2 Algoritmo Genético Generacional 12
3.3 Algoritmo Genético Estacionario 14
3.4 Algoritmo Memético 15
3.4.1 AM-Random 17
3.4.2 AM-All 17
3.4.3 AM-Best 18
4 Estructura del código 18
4.1 Organización de archivos 18
4.2 Compilación 19
4.3 Manual de usuario: Ejecución 20
5 Experimentos y análisis de los resultados 21
5.1 Casos empleados y valores de los parámetros. 21
5.2 Resultados obtenidos 21
5.3 Boxplots de los resultados 23
5.4 Gráficas de evolución del fitness durante las ejecuciones 24
6 Análisis de los resultados 26
6.1 Búsqueda Local vs Algoritmos Genéticos 26
6.2 Mejor operador para los genéticos 26
6.3 Comportamiento de los Algoritmos Genéticos Generacionales 27
6.4 Comportamiento de los Algoritmos Genéticos Estacionarios 27
6.5 AGE-BLα vs AGG-BLα 27
6.6 Comparación entre meméticos 28
6.7 Mejor Memético vs mejor AG 28
6.8 Análisis resultados 2025 29
6.9 Conclusiones 29

1 Descripción del problema

El problema que se ha escogido es el problema del portfolio, en el cual se nos da una lista de empresas de las cuales sabemos su retorno obtenido diario desde principios de 2015 hasta finales de 2025. Con estos datos necesitamos elaborar una solución indicando qué porcentaje de nuestra cartera deberíamos haber invertido en cada empresa para maximizar el beneficio de las inversiones.

En este contexto, el problema no solo busca el máximo retorno, sino que también suele considerar la correlación entre activos, teniendo esto en cuenta en las ecuaciones que se usan para maximizar el beneficio.

De esta manera podemos ajustar el programa ajustando el parámetro λ de la ecuación a maximizar para así tener mayor o menor aversión al riesgo.

2 Algoritmos empleados y estructura del problema

2.1 Estructura del problema

Para modelar el problema del portfolio se ha usado la clase Pportfolio la cual es hija de la clase Problem que viene ya predefinida con varios métodos virtuales, los cuales se implementan en la clase hija. Para representar el problema usando estas clases se tienen en cuenta los siguientes atributos:

- **days** : número entero de días de los que se tienen datos.
- **ncompany** : número entero de empresas de las que se tienen datos.
- **low** : mínimo porcentaje de inversión en una empresa (double).
- **high** : máximo porcentaje de inversión en una empresa (double).
- **lambda** : parámetro entero usado en el fitness.
- **data** : matriz de la librería Eigen con valores double que contienen los beneficios diarios de cada empresa, en las columnas se representan las empresas y en las filas los días.
- **covarianceMat** : matriz de la librería Eigen con valores double que contiene la covarianza de las empresas.
- **logSums** : vector de la librería Eigen con valores double que contiene la suma por columnas después de aplicar **log(valor+1)** a cada posición de la matriz.
- **ERROR** : Error permitido en la suma total del vector solución (double).

Podemos observar que se ha optado por usar la librería Eigen de C++ para poder manejar las operaciones de matrices y vectores como la suma total, la media total, la media por columnas o por filas, multiplicación escalar a escalar, etc. Esto hace que el código sea más compacto y fácil de seguir a la vez que más preciso y cómodo de utilizar.

También se han guardado los datos como **covarianceMat** y **logSums** que son constantes durante la ejecución del programa y solo cambian cuando cambia el archivo de entrada. Al ser valores calculados con operaciones matriciales son costosos de calcular y eso hace que el programa sea más rápido.

2.2 Representación de las soluciones

Para este problema las soluciones se representan por vectores de la biblioteca Eigen con tipo de dato double donde cada posición del vector representa la i-ésima empresa del fichero de datos de entrada, todos estos vectores deben cumplir las siguientes restricciones:

- El tamaño de la solución debe ser el mismo que el número de empresas en los datos de entrada.
- La suma de todos sus valores debe ser igual a 1.
- Cada posición del vector tiene que tener un valor entre **low** y **high**
- Puede haber posiciones del vector con valor 0 (representando que en esa empresa no se invierte).

La mejor solución será aquella que consiga maximizar la función objetivo.

2.3 Funciones comunes usadas por los algoritmos

En primer lugar la función que permite cargar los datos en nuestra clase Pportfolio es una función implementada en un archivo auxiliar ya que no tiene por qué pertenecer necesariamente al problema que se plantea. Esta función lee datos de un fichero en formato CSV. Tras abrir el archivo la función descarta la primera línea con los nombres de las empresas y comienza a leer las siguientes líneas, una vez escogida una línea, se salta el string que contiene la fecha y comienza a insertar datos al vector usando como separador la coma. El código comprueba los valores de la fecha leída, si el parámetro only2025 es True, solo se leen datos de 2025, de lo contrario se leen todos los datos hasta el último día del 2024. Una vez leídos todos los datos, la matriz obtenida se convierte a una matriz Eigen para poder manejarla con más facilidad.

```
funcion LoadCSV(archivo, only2025):
  matrix data
  leer(primera_linea)
  abrir archivo
  while(leer(linea_archivo)):
    if es fecha el primer string :
      continue
    if(es fecha el primer string and not only2025):
      solo leer valores de 2014-2024
    if(es fecha el primer string and only2025):
      solo leer valores de 2025
    vector valores

     while(leer(dato)):
       if(dato no vacío):
         valores.pushback(dato)
       else:
         valores.pushback(0.0)

    data.pushback(valores)

  cerrar archivo

   if(data no vacío):
     matriz_eigen = data
  return matriz_eigen
```

Las principales funciones que se usan de la clase **Pportfolio** para implementar los algoritmos son las siguientes:

La función **createSolution** se usa en dos algoritmos, en la Búsqueda Aleatoria para generar soluciones aleatorias y escoger finalmente la que mejor fitness tenga y en la Búsqueda Local para generar la solución desde la cual se comienza a buscar la mejor solución posible.

```
funcion createSolution():
   vector solución;

   for i<solución.size :
```

```
      solución(i) = Aleatorio entre high y low


    numeroceros = Aleatorio entre 1 y solución.size-1
    vectorindices = {0,1,2, ......,solución.size-1}
   desordenar(vectorindices)
    ## Seleccionamos valores aleatorios que ponemos a 0
    for i<numeroceros:
      solución(vectorindices[i])=0

    solución = solución/solución.sum
    if la solución no es válida
      fix(solución)

    return solución;
```

```
# Pseudocódigo del arreglo de solución
 funcion fix(solución):
    solución = solución/solución.sum

 for i in ncompany:
   if solución(i) < low:
     solución(i) = 0
   else if solución(i) > high:
     solución(i) = this->high


 double diff = 1.0 - solución.sum
 while diff >ERROR and no supera máximo de iteraciones:
  ajustado = false

   while i < ncompany and not ajustado:
     if solución(i) en rango [low, high):
       espacio = high - solución(i)
      if hay espacio :
        sumar diferencia y ajustado=true
       else :
         solución(i) = high
        diff =diff- espacio

   if no se ha ajustado y diferencia menor que low:
     for i in ncompany and not ajustado:
       if solución(i) == 0:
         solución(i) = mínimo entre diff y high
        ajustado = true

   if no se ha ajustado y la diferencia es menor que low:
     while i < ncompany and not ajustado:
       if (solución(j) > low and solución(j) + diff < high:
         solución(j) += diff
        ajustado = true

   diff = 1.0 - solución.sum
```

La solución fix se aplica cada vez que se genera una solución aleatoria para arreglarla, lo que se hace es tras normalizar la solución modificar los valores fuera de rango para que cumplan las condiciones de ser solución. Una vez hecho esto, si hace falta añadir más valor en alguna posición se añade, primero se comprueba si se puede repartir dentro de pesos que ya estén activos, de no ser así si la diferencia es mayor que el límite inferior se activa un cero de la solución. En caso de que la diferencia sea menor que el mínimo permitido se intenta distribuir la diferencia entre valores que ya estén activos y tengan un margen para ser de nuevo ajustados. Este proceso se repite hasta que la diferencia es menor que el error permitido o se sobrepasen las evaluaciones. En el código las evaluaciones permitidas son altas para que no haya problema por la precisión de las operaciones.

```
# Pseudocódigo del cálculo del beneficio
 funcion profit(solution):
   return (solution * logSums).sum #multiplicación componente a componente
```

```
# Pseudocódigo del cálculo del riesgo
 funcion risk(solution):
   return (solution.transpose * covarianceMat * solution);
#devuelve un número resultado de las dimensiones 1xn * nxn * nx1
```

```
# Pseudocódigo del cálculo del fitness
 funcion fitness(solution):
   return (profit - lambda*risk);
#devuelve un número resultado de las dimensiones 1xn * nxn * nx1
```

El principal objetivo de los algoritmos es maximizar la función fitness, la cual trata de maximizar el beneficio y minimizar el riesgo en una sola fórmula. El parámetro lambda se usará a modo de ajuste, cuando más alto sea su valor el algoritmo valorará mejor soluciones con poco beneficio pero que sean seguras, es decir, que tengan poco riesgo, mientras que cuanto más cercano a 0 sea lambda, el algoritmo valorará mejor las soluciones con máximo beneficio posible, ignorando el riesgo.El valor que se usa durante la práctica es $\lambda = 500$ que es el indicado en el guión.

2.4 Operadores

Para agrupar los operadores usados en los diferentes algoritmos he utilizado una clase **Operator** la cual tiene como métodos públicos los operadores usados en los algoritmos genéticos y meméticos.

2.4.1 Operadores de cruce

En este problema se han utilizado dos operadores de cruce, **Aritmético** y **BLX-α**

En ambos operadores, si los padres seleccionados son iguales, los hijos serán iguales a los padres.

El operador aritmético genera un aleatorio entre 0 y 1, para cada posición **i** de los padres, con el aleatorio escogido hace una combinación lineal distinta para cada hijo entre los genes de los padres. Si los genes de los padres son iguales en una determinada posición,

no se genera la transformación, de esta manera conseguimos que si todos los genes de los padres son iguales, los hijos sean iguales.

Al ser este cruce una combinación lineal, los genes de los hijos resultantes, nunca podrán tener un valor menor que el menor gen de los padres ni mayor que el mayor de los genes de sus padres. Esto hace que cuando se aplica este operador tienda a converger hacia el centro de los padres, con un comportamiento explotativo.

```
# Pseudocódigo del operador aritmético
 funcion arithmetic(padre1 , padre2):
   hijo1(padre1.size) ## genero los hijos con el mismo tamaño que los padres
   hijo2(padre1.size) ## en el problema que tratamos todos los cromosomas
miden lo mismo

     for i in range(0,tamaniopadres):

     if (padre1[i] == padre2[i]):
     # asigno el mismo gen a los hijos
     # de esta forma si los padres son idénticos
     # sus hijos también los son
     hijo1[i]=padre1[i]
     hijo2[i]=padre1[i]
     else:
     sigma = double aleatorio entre 0 y 1
     hijo1[i] =sigma*padre1[i]+(1-sigma)*padre2[i]
     hijo2[i] =sigma*padre2[i]+(1-sigma)*padre1[i]
    return pareja de hijos (hijo1, hijo2)
```

El operador BLX-alpha genera un intervalo entre el mínimo y el máximo de los genes seleccionados de los padres, a este intervalo se le añade un parámetro **alpha*gap**, el cual permite que el intervalo se agrande según el **alpha** escogido y la distancia entre el mínimo y el máximo. Esto permite que el valor aleatorio que se genere en ese intervalo para cada hijo, pueda ser más grande o más pequeño que los valores que los padres tenían, por lo que permite explorar más soluciones y mantiene más diversidad ya que se generan genes que están fuera de los rangos de los padres.

```
# Pseudocódigo del operador BLX-alpha
 funcion arithmetic(padre1 , padre2):
   hijo1(padre1.size) ## genero los hijos con el mismo tamaño que los padres
   hijo2(padre1.size) ## en el problema que tratamos todos los cromosomas
miden lo mismo

    for i in range(0,tamaniopadres):

    if(padre1[i] == padre2[i]):
     # asigno el mismo gen a los hijos
     # de esta forma si los padres son idénticos
     # sus hijos también los son
     hijo1[i]=padre1[i]
     hijo2[i]=padre1[i]
    else:
      cmax = double máximo entre padre1[i],padre2[i]
      cmin = double mínimo entre padre1[i],padre2[i]
```

```
    gap = cmax-cmin
    hijo1[i] = aleatorio entre (cmin-gap*alpha,cmax+gap*alpha)
    hijo2[i] = aleatorio entre (cmin-gap*alpha,cmax+gap*alpha)
  return pareja de hijos (hijo1, hijo2)
```

Ambos operadores devuelven parejas de hijos que pueden no ser válidos, por lo que se tendrán que arreglar ambos cuando estos se usen en los correspondientes algoritmos.

2.4.2 Operador de selección

Para la selección de los padres se usa un torneo entre k cromosomas elegidos al azar de la población actual. Estos k cromosomas compiten por ver quién tiene mejor fitness y el que gane se convierte en padre. La función recibe como parámetros el número de seleccionados por torneo (k), el tamaño de la población, el número de padres que se deben de seleccionar y el vector de fitness de la población, para economizar las evaluaciones. Como resultado la función devuelve un vector de índices de los cromosomas escogidos para ser padres.

```
# Pseudocódigo del operador tournament
  funcion tournament(tournamentsize, populationsize, selectionsize,
fitnesspoblation):
  vector idxparents # vector con los índices de los padres
  # repito el torneo tantas veces como necesite
      for i in range(0, selectionsize):
      idxbest = 0

      for j in range(0, tournamentsize):
        idx = aleatorio entre 0 y populationsize-1
      #selecciono el mejor de entre los participantes del torneo
        if fitnesspoblation[idx] > fitnessTournament:
          fitnessTournament = fitnesspoblation[idx]
          idxBest = idx
      #añado el mejor del torneo a la lista de padres
      idxparents.pushback(idxBest)
  return idxparents
```

2.4.3 Operador de mutación

A la hora de hacer la mutación en los algoritmos usaremos el siguiente método:

```
# Pseudocódigo del operador de mutación
funcion mutation(mutSol, problem, mutationRate, maxAttempts):
  problembounds = problem.getSolutionDomainRange()
  solutionsize = problem.getSolutionSize()

  validDonors = []
  #busco un donante valido
  for i in range(0, solutionsize):
    amount = mutSol(i) * mutationRate
    if amount <= 0.0:
      continuar
    firstAfter = mutSol(i) - amount
```

```
    firstValid = firstAfter == 0.0 o
                 (firstAfter >= problembounds.first y
                  firstAfter <= problembounds.second)
    if firstValid:
      validDonors.pushback(i)

  if validDonors está vacío:
    return false
  # busco un receptor valido
  for attempt in range(0, maxAttempts):
    pickDonor = aleatorio entre 0 y validDonors.size - 1
    candidateFirst = validDonors[pickDonor]
    amount = mutSol(candidateFirst) * mutationRate

    validReceivers = []
    for i in range(0, solutionsize):
      if i == candidateFirst:
        continuar
      secondAfter = mutSol(i) + amount
      secondValid = secondAfter == 0.0 o
                    (secondAfter >= problembounds.first y
                     secondAfter <= problembounds.second)
      if secondValid:
        validReceivers.pushback(i)

    if validReceivers está vacío:
      continuar

    idx = aleatorio entre 0 y validReceivers.size - 1
    candidateSecond = validReceivers[idx]

    mutSol(candidateFirst) -= amount
    mutSol(candidateSecond) += amount
    return true

  return false
```

La función de mutación recibe como parámetros la solución a mutar (por referencia), un objeto problema, el ratio de mutación y el número máximo de intentos que tenemos para elegir el segundo gen.

Como se puede observar, el proceso de mutación se divide en dos partes:

- En el primer bucle se buscan todos los genes a los cuales se les puede restar el porcentaje indicado en la variable **mutationrate** y que siguen cumpliendo las restricciones del problema.Si no se encuentran posibles donantes se acaba la ejecución.
- En el segundo bucle, se selecciona un gen donante del vector calculado previamente y se buscan todos los posibles genes que puedan recibir la cantidad que se le va a restar a este gen.Si no se encuentra ningún gen que pueda recibir dicha cantidad , se vuelve a escoger un donante aleatorio y se repite el proceso. Este proceso se repite todas las veces que permita el limite de intentos marcado por **maxattempts** . En la práctica , tras varias pruebas el límite se ha puesto lo suficientemente alto para que se pueda encontrar el gen receptor.

3 Estructura de los algoritmos

3.1 Local Search

Es el mismo que la práctica anterior , pero al aplicarlo modificaremos el ratio y la solución inicial que toma.

El algoritmo de búsqueda local (el primero el mejor) parte de una solución generada aleatoriamente y genera un vecindario alrededor de esta solución, una vez generado el vecindario, se comienza a recorrer cada vecino y se evalúa su fitness. Si se encuentra un fitness mejor que el de la solución actual, se escoge ese vecino como mejor solución y se genera un nuevo vecindario alrededor de él. El proceso termina cuando se llega al máximo de evaluaciones permitidas o cuando no se ha encontrado ningún vecino del vecindario actual generado que mejore la solución.

```
# Pseudocódigo del algoritmo de búsqueda local
función LocalSearch(max_evals):
    double ratio
     solucioninicial = solucionAleatoria
     bestFitness = fitness(solucionAleatoria)

    vecinos = Generar vector de todas las parejas (i,j) posibles con i,j en
[0,ncompany-1]
    # excepto en el caso i=j;

    while evals < maxevals and haymejora:

      mezclar vecinos
      for v in vecinos:
        cantidadmover = ratio*solucioninicial[v.primero]

        solucioninicial[v.primero]-=cantidadmover
        solucioninicial[v.segundo]+=cantidadmover

         if es válida solucioninicial:
          evals = evals+1

           if fitness(solucioninicial) > mejorfitness:
             haymejora = true
             mejorfitness = fitness de la solución inicial modificada
             #si hay una mejora salimos del bucle for
             #ya que es una búsqueda del primero el mejor
            salir del bucle for
         # si no cambiamos de solución inicial, hay que
         # devolverla a su estado original
         devolver solucioninicial a su estado original
         if evals==maxevals:
          salir del bucle for
          #para no pasarnos de evaluaciones
     return solucioninicial, mejorfitness, evals
```

```
  # Generación del vecindario
 for i,j in ncompany:
   vectorparejas.insertar(pareja(i,j))
```

La implementación del algoritmo de Búsqueda local se divide en dos partes, primero se crea el vector vecinos, en este caso se usa un vector de parejas, donde cada vecino es la pareja (i,j) la cual nos indicará los índices que debemos modificar en el vector solución. Una vez creado el vector, lo mezclamos aleatoriamente, y este será el que nos indique qué vecinos debemos de probar. Si no fuese una búsqueda del primero el mejor y se escogiese el mejor vecino no haría falta mezclar las parejas, ya que el orden no importaría porque siempre tendríamos que recorrer todo el vecindario.

Una vez disponemos del vecindario, debemos generar las nuevas soluciones a partir de las parejas que vamos escogiendo del vector. Si tenemos la pareja (i,j) quitaremos un porcentaje del peso a la posición i y se la sumaremos a la posición j (el ratio usado en la práctica es 0.4). Esto nos genera una nueva solución la cual comprobaremos si es válida y, en caso de ser válida, si supera el fitness de nuestra mejor solución hasta el momento. Este nuevo vector solución con las nuevas modificaciones en los índices (i,j) se convertirá en nuestra nueva solución. Dejaremos de recorrer el vector de vecinos y volveremos a generar un vecindario mezclado aleatoriamente para recorrerlo de nuevo desde el principio para ver si a partir de esta nueva solución generada conseguimos una mejor.

Si la solución generada no es válida o no mejora la solución actual, restauraremos las modificaciones hechas al vector solución. De esta manera podemos usar un solo vector y modificar dos posiciones en vez de hacer copias de vectores completos.

Mientras que no se superen las restricciones de las evaluaciones y se sigan encontrando vecinos mejores en cada vecindario seguiremos intentando encontrar la mejor solución. En caso de recorrer todo el vecindario y no encontrar una solución mejor o que se superen las evaluaciones permitidas, se detendrá la búsqueda.

3.2 Algoritmo Genético Generacional

Para el algoritmo genético generacional se ha construido una clase **Agg** para implementar la estructura del algoritmo y dos clases hijas **AggArit** y **AggBLX**, las cuales heredan de **Agg** y solo modifican la función de cruce. De esta manera podemos tener una única implementación del algoritmo y cambiar el operador de cruce con facilidad.

```
# Pseudocódigo del algoritmo genético generacional (AGG)
función Agg(max_evals):
    evals = 0
    bestFitness = -infinito
    bestSolution = vacio
    poblation = vector de soluciones aleatorias #tamaño populationSize
    for i in range(0, populationSize):
      tempfitness = evaluar fitness de poblation[i]
      evals += 1
      añadir tempfitness a fitness de la población
      if tempfitness > bestFitness:
        bestFitness = tempfitness
```

```
        bestSolution = poblation[i]

    while evals < maxevals:
      idxparents = torneo(tournamentsize, populationsize, populationsize,
fitness de la población)
      expectedparentscrossover = redondeo hacia arriba(crossoverprob *
idxparents.size())
      sons = [] # poblacion nueva de hijos
      for i in range(0, expectedparentscrossover, 2):
        son = cruce(poblation[idxparents[i]], poblation[idxparents[i + 1]])
        reparar hijos inválidos
        añadir hijos a la población nueva

      for i in range(expectedparentscrossover, populationsize):
        #añado los padres que no se han cruzado a la población actual
        añadir poblation[idxparents[i]] a sons
      ## mutación
      expectedmutation = mutationprob * populationsize
      for i in range(0, expectedmutation):
        idxsolution = random entre 0 y populationsize - 1
        mutsol = sons[idxsolution]
        mutar mutsol con parámetros del algoritmo

        sons[idxsolution] = mutsol

      # Sustitución de la población

      bestfitnessSons = -infinito
      worstfitnessSons = infinito
      for i in range(0, populationsize - 1):
        tempfitness = evaluar fitness de sons[i]
        evals += 1
        añadir tempfitness a fitness de la población
        if tempfitness > bestfitnessSons:
          selecciono mejor hijo
        if tempfitness < worstfitnessSons:
          selecciono peor hijo
        if evals == maxevals:
          terminar
      if bestFitness > worstfitnessSons:
        introduzco el mejor de la población anterior
        a la nueva
      if bestfitnessSons > bestFitness:
        actualizo mejor fitness y mejor solucion
      poblation = sons
  return bestsolution, bestfitness, evals
```

El algoritmo comienza seleccionando individuos de la población mediante un torneo en el que participan k individuos. El torneo se aplica tantas veces como individuos de la población haya. Una vez se obtienen los ganadores de cada torneo, se seleccionan la cantidad indicada por la probabilidad de cruce para que se crucen entre ellos. De cada cruce surgen dos hijos que serán insertados en la nueva generación (vector sons). Una vez se terminan de cruzar los individuos seleccionados, se terminan de añadir a la nueva generación los que no han participado en el cruce. Una vez ya se tiene la nueva generación,

se aplica la mutación escogiendo soluciones aleatorias de la población y mutándolas en función de la probabilidad indicada. Tras la mutación se realiza el cambio generacional donde conservamos únicamente la solución de la población anterior. Esta se cambia por la peor solución de la nueva población en caso de que supere a la peor solución. Si en la nueva población se encuentra una solución mejor que la actual, se actualiza.

3.3 Algoritmo Genético Estacionario

Para el algoritmo genético estacionario se ha construido una clase **Age** de la cual heredan las clases **AgeArti** y **AgeBLX-α**, al igual que en los AGG, estas clases solo modifican el método crossover.

```
# Pseudocódigo del algoritmo genético estacionario (AGE)
función Age(maxevals):
    evals = 0
    bestFitness = -infinito
    bestSolution
    poblation = soluciones aleatorias
    for i in range(0, populationSize):
      tempfitness = evaluar fitness de poblation[i]
      evals += 1
      añadir tempfitness a fitness de la población
      if tempfitness > bestFitness:
        bestFitness = tempfitness
        bestSolution = poblation[i]

    while evals < maxevals:
      idxparents = torneo(tournamentsize, populationsize, 2, fitness de la
población)

      ## cruce
      son = cruce(poblation[idxparents[0]], poblation[idxparents[1]])
      reparar hijos inválidos

      ## mutación
      for i in range(0, 2):
        if probabilidad de mutar:
          mutar hijo con parámetros del algoritmo

      # evaluación de los hijos
      fitnessFirst = evaluar fitness de son[0]
      evals += 1
      if evals == maxevals:
        terminar
      fitnessSecond = evaluar fitness de son[1]
      evals += 1
      if evals == maxevals:
        terminar

      # ordenar hijos por fitness
      if fitnessSecond > fitnessFirst:
        intercambiar(fitnessFirst, fitnessSecond)
        intercambiar(son[0], son[1])
```

```
      # localizar peor y segundo peor
      localizar índices del peor y segundo peor en fitness de la población

      # actualizar mejor solución global
      if fitnessSecond > bestFitness:
        actualizo mejor solucion con los valores de son[1]

      if fitnessFirst > bestFitness:
        actualizo mejor solucion con los valores de son[0]

      # reemplazo estacionario
      usado = false
      if fitnessFirst > worstFitness:
        reemplazar peor por son[0]
      else if fitnessSecond > worstFitness:
        reemplazar peor por son[1]
        usado = true

      if not usado and fitnessSecond > secondWorstFitness:
        reemplazar segundo peor por son[1]

  return bestsolution, bestfitness, evals
```

Este algoritmo selecciona dos padres de la población mediante el torneo. Estos padres se cruzan y se obtienen dos hijos a los que se les aplica la mutación en función de la probabilidad indicada por el parámetro **MutationProb**. Tras la mutación, calculo el fitness de ambos hijos y compruebo si alguno mejora la mejor solución actual; si es así, actualizo los valores. A la hora de insertarlos en la población, se buscan el primer y el segundo peores de la población actual; si los hijos son mejores, se introducen. También puede darse la situación de que un hijo entre y el otro no.

3.4 Algoritmo Memético

Los algoritmos meméticos se han implementado hibridando el **AGG-BLXα** con la ejecución de la búsqueda local cada 10 generaciones de poblaciones.La estructura del código es idéntica a la expuesta en el AGG pero con una condición adicional al final del bucle principal.

Una pequeña diferencia que se puede encontrar respecto a la implementación es que la variable evals pertenece a la clase **Am** para poder seguir la cuenta del número de iteraciones al aplicar la búsqueda local.

```
# Pseudocódigo del algoritmo memético (AM)
función Am(max_evals):
    evals = 0
    bestFitness = -infinito
    bestSolution = vacio
    poblation = vector de soluciones aleatorias #tamaño populationSize
    for i in range(0, populationSize):
      tempfitness = evaluar fitness de poblation[i]
      evals += 1
```

```
      añadir tempfitness a fitness de la población
      if tempfitness > bestFitness:
        bestFitness = tempfitness
        bestSolution = poblation[i]

    while evals < maxevals:
      idxparents = torneo(tournamentsize, populationsize, populationsize,
fitness de la población)
      expectedparentscrossover = redondeo hacia arriba(crossoverprob *
idxparents.size())
      sons = [] # poblacion nueva de hijos
      for i in range(0, expectedparentscrossover, 2):
        son = cruce(poblation[idxparents[i]], poblation[idxparents[i + 1]])
        reparar hijos inválidos
        añadir hijos a la población nueva

      for i in range(expectedparentscrossover, populationsize):
        #añado los padres que no se han cruzado a la población actual
        añadir poblation[idxparents[i]] a sons
      ## mutación
      expectedmutation = mutationprob * populationsize
      for i in range(0, expectedmutation):
        idxsolution = random entre 0 y populationsize - 1
        mutsol = sons[idxsolution]
        mutar mutsol con parámetros del algoritmo

        sons[idxsolution] = mutsol

      # Sustitución de la población

      bestfitnessSons = -infinito
      worstfitnessSons = infinito
      for i in range(0, populationsize - 1):
        tempfitness = evaluar fitness de sons[i]
        evals += 1
        añadir tempfitness a fitness de la población
        if tempfitness > bestfitnessSons:
          selecciono mejor hijo
        if tempfitness < worstfitnessSons:
          selecciono peor hijo
        if evals == maxevals:
          terminar
      if bestFitness > worstfitnessSons:
        introduzco el mejor de la población anterior
        a la nueva
      if bestfitnessSons > bestFitness:
        actualizo mejor fitness y mejor solucion
      poblation = sons


      # aplicación de la búsqueda local
      nGenerations += 1
      if nGenerations % 10 == 0:
        applyLocalSearch
```

```
  return bestsolution, bestfitness, evals
```

Para la implementación de los algoritmos meméticos especificados he usado el método virtual applyLocalSerch de la clase **Am** para poder cambiar el modo de aplicar la búsqueda local sin tener que modificar el código general.

En los siguientes apartados se encuentra la descripción de la función **applyLocalSearch** de cada algoritmo.

La función **localSearch** contiene el mismo código que la búsqueda local descrita anteriormente , la única modificiación que se le hace es que se le pasa como parámetro la solución inicial que debe usar.

3.4.1 AM-Random

```
# Pseudocódigo de AM-Random (applyLocalSearch)
función applyLocalSearch(poblation, fitnesspoblation, bestFitness,
bestsolution, problem,maxevals):
    nRand = redondeo hacia arriba(poblation.size() * 0.10)
    for i in range(0, nRand):
      if evals+100>maxevals:
        termina el bucle
      else : evals+=100
      result = localSearch(problem, 100, poblation[i])
      if result.fitness > fitnesspoblation[i]:
        actualizo fitness y solución en la posición i
      if result.fitness > bestFitness:
        actualizo mejor fitness y mejor solución
```

En este algoritmo se aplica la BL a un 10% de la población escogida aleatoriamente , si los resultados de la búsqueda local superan a los cromosomas escogidos , se insertan en su respectiva posición. Si se consigue una nueva mejor solución, también se actualiza.

No necesitamos generar números aleatorios ya que el proceso de generación de la población ya aleatoriza las posiciones en las que se encuentran los cromosomas.

3.4.2 AM-All

```
# Pseudocódigo de AM-All (applyLocalSearch)
función applyLocalSearch(poblation, fitnesspoblation, bestFitness,
bestsolution, problem):
    for i in range(0, populationSize):

      if evals+100>maxevals:
        termina el bucle
      else : evals+=100
      result = localSearch(problem, 100, poblation[i])
      if result.fitness > fitnesspoblation[i]:
        actualizo fitness y solución en la posición i
      if result.fitness > bestFitness:
        actualizo mejor fitness y mejor solución
```

En este algoritmo se aplica la BL a toda la población, si los resultados de la búsqueda local superan a los cromosomas escogidos , se insertan en su respectiva posición. Si se consigue una nueva mejor solución, también se actualiza.

3.4.3 AM-Best

```
# Pseudocódigo de AM-Best (applyLocalSearch)
función applyLocalSearch(poblation, fitnesspoblation, bestFitness,
bestsolution, problem):
    evals = 0
    nBest = redondeo hacia arriba(populationSize * 0.10)
     indices = índices de las nBest mejores soluciones según fitnesspoblation
      reduzco indices a un tamaño nBest para quedarme con los nBest mejores
    for i in indices:
      if evals+100>maxevals:
        termina el bucle
      else : evals+=100
      result = localSearch(problem, 100, poblation[i])
      if result.fitness > fitnesspoblation[i]:
        actualizo fitness y solución en la posición i
      if result.fitness > bestFitness:
        actualizo mejor fitness y mejor solución
```

En este algoritmo se aplica la BL al 10% de las mejores soluciones que están en la población. Para ello primero genero un vector con los índices que índican donde se encuentra el fitness de cada individuo , y los ordeno de mayor a menor dependiendo de su fitness.Selecciono el 10% inicial y aplico la búsqueda local sobre ellos , si la BL mejora los resultados desde los que empezó se sustituyen y si mejora la mejor solución la actualizo.

4 Estructura del código

El código de la práctica está organizado en una estructura modular que facilita la compilación y ejecución. A continuación se describe la organización del proyecto y el manual de usuario para compilar y ejecutar el programa.

4.1 Organización de archivos

El proyecto está organizado de la siguiente manera:

- `software/` : Directorio raíz que contiene todo el código fuente
  - `main.cpp` : Archivo principal que ejecuta los algoritmos
  - `src/` : Contiene las implementaciones (.cpp)
    - `age.cpp` : Implementación del algoritmo AGE
    - `agg.cpp` : Implementación del algoritmo AGG
    - `am.cpp` : Implementación del algoritmo AM
    - `amall.cpp` : Implementación de applyLocalSearch para amAll
    - `ambest.cpp` : Implementación de applyLocalSearch para amBest
    - `amrand.cpp` : Implementación de applyLocalSearch para amRand
    - `pportfolio.cpp` : Implementación de la clase del problema

  - `randomsearch.cpp` : Implementación del algoritmo de búsqueda aleatoria
  - `greedy.cpp` : Implementación del algoritmo Greedy
  - `localsearch.cpp` : Implementación del algoritmo de búsqueda local
  - `operator.cpp` : Implementación de la clase Operator
- `include/` : Contiene los archivos de cabecera (.h y .hpp)
  - `age.h`: Clase base para los algoritmos genéticos estacionarios
  - `agg.h`: Clase base para los algoritmos genéticos generacionales
  - `am.h`: Clase base para los algoritmos meméticos
  - `agearit.h`, `ageblx.h`: Clases hijas de `age.h` con los correspondientes operadores
  - `aggarit.h`, `aggblx.h`: Clases hijas de `agg.h` con los correspondientes operadores
  - `amall.h`, `ambest.h`, `amrand.h`: Clases de los algoritmos meméticos.
  - `pportfolio.h`, `greedy.h`, `localsearch.h`, `randomsearch.h`
  - `operator.h` : Clase de operadores genéticos
  - `problem.h` : Clase base abstracta
  - `mh.h` : Clase base para los algoritmos de metaheurísticas
  - `solution.h` : Estructura para representar soluciones
  - `random.hpp`, `utils.hpp` : Utilidades auxiliares
  - `Eigen/` : Biblioteca para operaciones con matrices y vectores
- `data/` : Contiene los archivos CSV con los datos de entrada
- `CMakeLists.txt` : Archivo de configuración de CMake
- `script.sh` : Script de compilación automatizada
- `execute_all.sh` : Script para ejecutar el programa en los 3 dataset a la vez.
- `Makefile` : Archivo generado automáticamente por CMake

4.2 Compilación

El proyecto utiliza **CMake** como sistema de compilación automática. Se proporcionan dos modos de compilación:

Opción 1: Usando el script de compilación

El script **script.sh** automatiza el proceso de compilación:

```
# Para compilar en modo debug (con símbolos de depuración):
./script.sh debug

# Para compilar en modo release (optimización máxima):
./script.sh release
```

- **Modo debug**: Compila con la flag -O0 (sin optimización) y -g (símbolos de depuración), para detectar errores y depurar.
- **Modo release**: Compila con -O3 (máxima optimización) y -DNDEBUG (desactiva comprobaciones de Eigen), con esta compilación se han hecho las pruebas.

Opción 2: Usando CMake directamente

```
# Configurar el proyecto en modo release:
cmake CMakeLists.txt -DCMAKE_BUILD_TYPE=Release
```

```
# Compilar:
make
```

Tras cualquiera de estos procesos se generará el ejecutable **program** en el directorio software/.

4.3 Manual de usuario: Ejecución

Una vez compilado el programa, se puede ejecutar de las siguientes formas:

Ejecución estándar (comparación de algoritmos):

```
./program <ruta_archivo_csv> <low> <high> <evaluaciones> <seed> <output_csv>
```

Parámetros:

- **ruta_archivo_csv**: Ruta al archivo CSV con los datos de retornos diarios (ejemplo: data/portfolio.csv)
- **low**: Porcentaje mínimo de inversión permitido por empresa (ejemplo: 0.005)
- **high**: Porcentaje máximo de inversión permitido por empresa (ejemplo: 0.08)
- **evaluaciones**: Número de ejecuciones de cada algoritmo con diferentes seeds
- **seed**: Semilla inicial para el generador de números aleatorios (se incrementa en 10 por cada evaluación)
- **output_csv** : Archivo CSV donde se guardan los fitness de todas las ejecuciones

El parámetro output_csv es opcional, si no se pone entonces no se guardan los datos en un fichero y solo aparecen las medias.

Ejemplo de ejecución:

```
./program data/syp_100.csv 0.005 0.05 50 42 syp100.csv
```

Este comando ejecutará cada uno de los tres algoritmos (Random Search, Greedy y Local Search) 5 veces con seeds diferentes (42, 52, 62, 72, 82). Al finalizar mostrará para cada algoritmo:

- Fitness medio obtenido
- Fitness del año 2025
- Beneficio obtenido en el año 2025
- Número medio de evaluaciones realizadas
- Tiempo medio de ejecución (en segundos)

Al final creará un .csv con los datos de la ejecución

Ejecución simplificada (valores por defecto):

```
./program data/syp_100.csv 0.005 0.05
```

Ejecuta una sola evaluación con seed 42.

Ejecución de ayuda:

```
./program
```

Muestra el mensaje de ayuda con las instrucciones de uso.

5 Experimentos y análisis de los resultados

5.1 Casos empleados y valores de los parámetros.

Para comparar los resultados de los algoritmos se han usado los beneficios diarios de las empresas del S&P500, S&P100 e IBEX35 con los siguientes parámetros.

- $\boldsymbol{\lambda}$= 500 (usado en la función fitness)
- **seed** = 42 (por cada ejecución del algoritmo se incrementa 10 unidades)
- **low** = 0.005 (para todos los datos)
- **high**(para IB35) = 0.08
- **high**(para S&P100) = 0.05
- **high**(para S&P500) = 0.02
- **totalevaluations** = 50

La ejecución se ha hecho como se indicó en el apartado anterior, cambiando en los argumentos del programa el archivo de entrada de datos y ajustando su respectivo valor **high** correspondiente a cada archivo.

5.2 Resultados obtenidos

Algoritmo	**2015-2024 Fitness**	**2025 Fitness**	**Beneficio**	**Evaluaciones**	**Tiempo (segundos)**
Greedy	−4.558	−3.254	0.040	1	0.001
Random	−4.980	−3.378	0.052	10000	0.005
BL	−4.608	−3.115	0.020	1517.36	0.001
AGG-Arit	−4.657	−3.189	0.055	10000	0.002
AGG-BLX	−4.500	−3.119	0.044	10000	0.003
AGE-Arit	−4.576	−3.153	0.051	10000	0.003
AGE-BLX	−4.495	−3.101	0.039	10000	0.003
AM-ALL	−4.496	−3.085	0.031	10000	0.026
AM-RAND	−4.498	−3.086	0.031	10000	0.006
AM-BEST	−4.494	−3.084	0.032	10000	0.005

Tabla 1: Resultados de Ibex 35 (Medias)

Algoritmo	**2015-2024 Fitness**	**2025 Fitness**	**Beneficio**	**Evaluaciones**	**Tiempo (segundos)**
Greedy	−2.835	−3.691	0.092	1	0.001
Random	−3.408	−4.174	0.088	10000	0.02
BL	−3.118	−3.888	0.096	8984.76	0.01
AGG-Arit	−3.184	−3.947	0.097	10000	0.012
AGG-BLX	−2.866	−3.757	0.100	10000	0.014
AGE-Arit	−3.127	−3.901	0.096	10000	0.014
AGE-BLX	−2.870	−3.757	0.102	10000	0.012
AM-ALL	−2.8744	−3.761	0.096	10000	0.289
AM-RAND	−2.945	−3.782	0.098	10000	0.042
AM-BEST	−2.814	−3.755	0.101	10000	0.044

Tabla 2: Resultados de S&P100 (Medias)

Algoritmo	**2015-2024 Fitness**	**2025 Fitness**	**Beneficio**	**Evaluaciones**	**Tiempo (segundos)**
Greedy	−2.749	−3.776	0.013	1	0.001
Random	−3.638	−4.389	0.026	10000	0.253
BL	−3.517	−4.324	0.010	10000	0.220
AGG-Arit	−3.321	−4.203	−0.002	10000	0.189
AGG-BLX	−3.003	−4.030	0.012	10000	0.191
AGE-Arit	−3.338	−4.241	−0.001	10000	0.202
AGE-BLX	−3.095	−4.090	0.009	10000	0.184
AM-ALL	−3.219	−4.113	0.004	10000	2.687
AM-RAND	−3.225	−4.131	0.000	10000	0.446
AM-BEST	−3.220	−4.156	0.001	10000	0.456

Tabla 3: Resultados de S&P500 (Medias)

Los valores presentados en la tabla corresponden a las medias de 5 ejecuciones independientes con diferentes semillas aleatorias.

5.3 Boxplots de los resultados

A continuación se muestran los boxplots de los resultados obtenidos para cada conjunto de datos tras 50 ejecuciones de cada algoritmo:

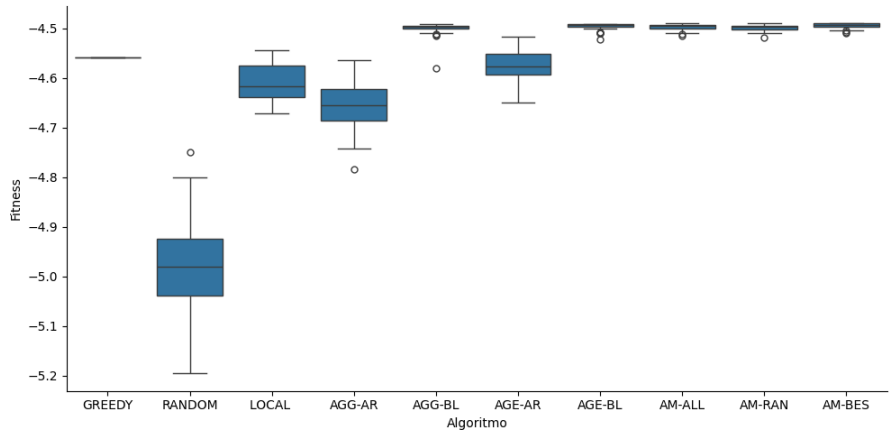


Figura 1: Boxplot de resultados para Ibex 35

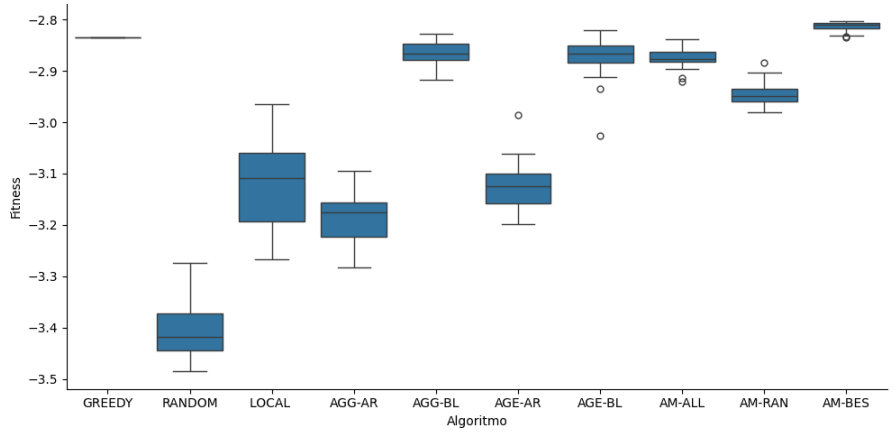


Figura 2: Boxplot de resultados para S&P100

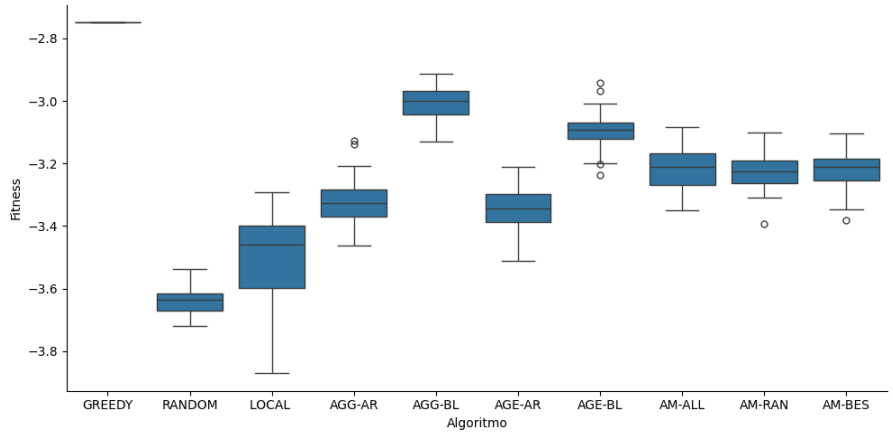


Figura 3: Boxplot de resultados para S&P500

5.4 Gráficas de evolución del fitness durante las ejecuciones

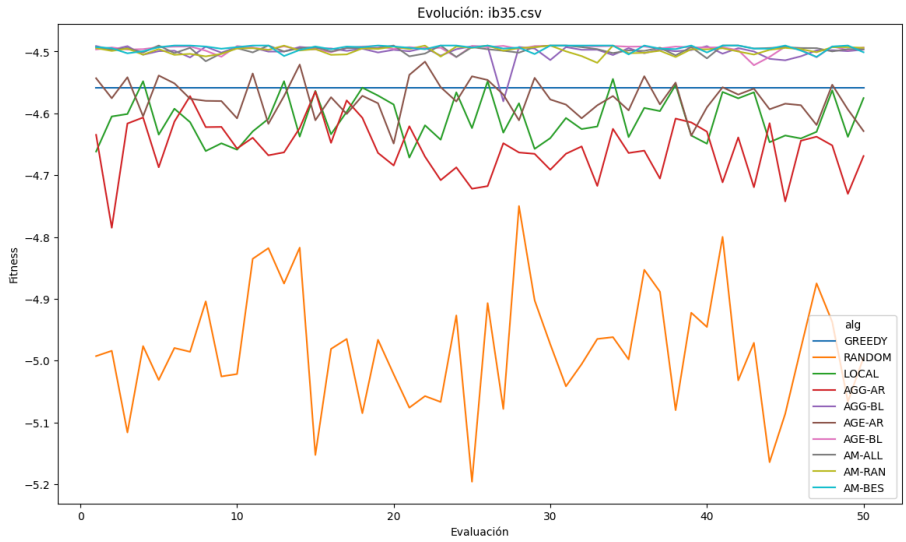


Figura 4: Comparación fitness para Ibex 35

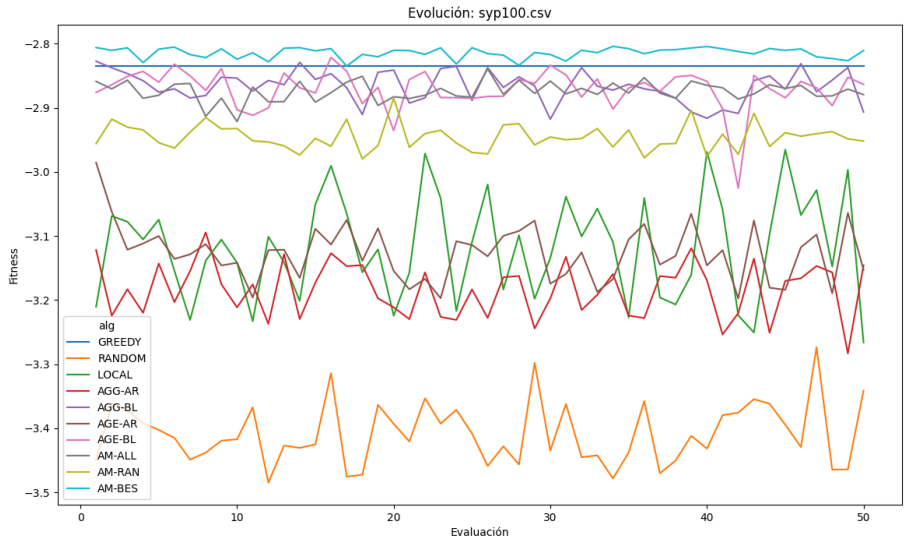


Figura 5: Comparación fitness para S&P100

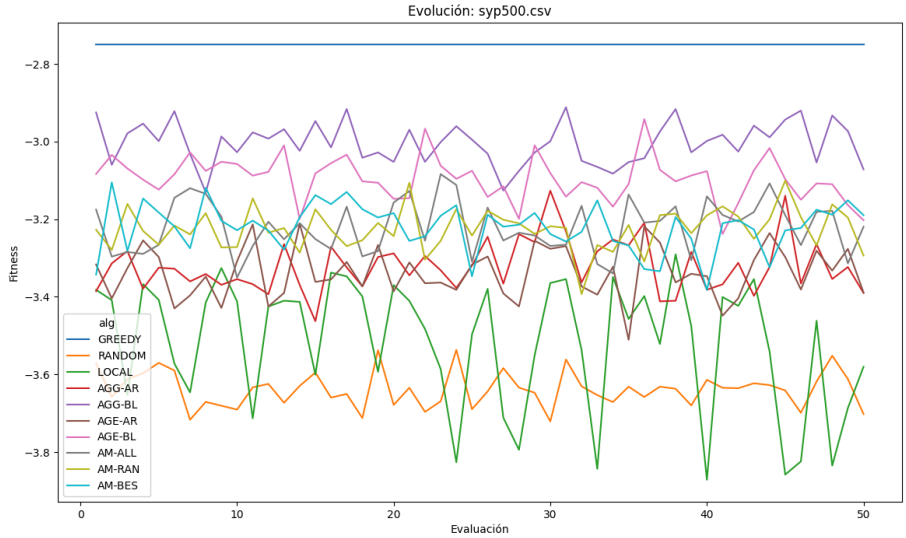


Figura 6: Boxplot de resultados para S&P500

En los gráficos de lineas podemos ver claramente la variabilidad del fitness en cada algoritmo , vemos que la búsqueda local y los algoritmos genéticos con el operador aritmético con los menos estables mientras que los genéticos con el operador BLX-α y los meméticos presentan una mayor estabilidad en su fitness a lo largo de las 50 ejecuciones.

6 Análisis de los resultados

Al observar los resultados obtenidos, podemos ver que cada algoritmo se comporta de manera muy diferente dependiendo del conjunto de datos usado. Para poder entender por qué unos algoritmos funcionan mejor que otros, es necesario analizar cómo trabaja cada uno y qué características del problema aprovechan o ignoran.

6.1 Búsqueda Local vs Algoritmos Genéticos

Los algoritmos genéticos en el problema del portfolio obtienen mejores resultados, ya que exploran más el espacio de soluciones que la búsqueda local. La búsqueda local es más explotativa, por lo que le es fácil entrar en máximos locales en datasets pequeños como el Ibex 35 (podemos ver que acaba antes que el máximo de evaluaciones), mientras que los algoritmos genéticos exploran un mayor rango de soluciones, por lo que tienen más facilidad de evitar máximos locales. También cabe destacar que la búsqueda local se ve muy afectada por la solución inicial que se toma; al ser esta aleatoria, dependemos más de la «suerte» a la hora de encontrar una mejor solución antes de las evaluaciones permitidas. Esto lo podemos ver claramente en los boxplots, donde la caja de la búsqueda local tiene el cuerpo mucho más grande que la de los otros algoritmos, debido a la variabilidad de la solución inicial escogida. En los algoritmos genéticos también influye la población inicial; a los AGG esto les influye en menor medida ya que, al reemplazar toda la población en cada iteración y conservar la mejor solución, se consigue una mayor diversidad y mayor probabilidad de no quedarse atascado en un máximo local.

Comparando los resultados de los fitness promedio y las gráficas de líneas, podemos ver que la búsqueda local en datasets más pequeños alcanza a los algoritmos genéticos que usan el operador aritmético, ya que este operador no da tanta diversidad en el cruce. Pero a medida que el tamaño del dataset aumenta, la búsqueda local se va quedando atrás y es fácilmente superada por los algoritmos genéticos. Donde más se nota la diferencia es en los que usan el operador BLX-α, que como veremos a continuación es el que mejores resultados obtiene.

6.2 Mejor operador para los genéticos

Como se puede observar en los resultados, en ambos algoritmos genéticos el mejor resultado lo obtiene el operador BLX-α. La principal diferencia entre este y el aritmético es que el BLX-α permite que el gen resultante de los hijos al cruzar los genes de los padres pueda estar fuera del intervalo que genera el mínimo y el máximo de los padres, por tanto los hijos resultantes de los cruces se pueden alejar más de sus padres y permiten explorar más el espacio de soluciones. Por otra parte, el cromosoma resultante del cruce aritmético contiene genes que no pueden tener un valor mayor o menor que el mayor o menor de los padres (al ser una combinación lineal) por lo que al aplicar este tipo de cruce, los hijos convergen a la media de los padres obteniendo mejor resultado en entornos explotativos.

Como consecuencia de estas propiedades, vemos que en la tabla del ibex35 la diferencia entre la media de los fitness no es tan alta, pero en cuanto pasamos a los dataset del sp100 y el sp500, al aumentar el espacio de búsqueda, se ven claramente más recompensados

los algoritmos que usan el operador BLX-α ya que exploran más parte del espacio de soluciones. Si nos fijamos en los boxplots, vemos que al usar el operador aritmético las cajas son más grandes; esto es debido a que, al generar hijos que se encuentran siempre en el intervalo de valores de los padres, la población inicial influye más en el resultado del algoritmo. Por otra parte, las cajas de los que usan el operador BLX-α están más comprimidas, sobre todo en espacios de búsqueda pequeños. Los outliers que presentan se encuentran sobre todo cuando se combina con el AGE, ya que la población inicial influye más.

6.3 Comportamiento de los Algoritmos Genéticos Generacionales

Los AGG reemplazan en todas las iteraciones la población al completo, conservando el mejor de la población anterior e intercambiándolo por el peor de la generación que entra. Este tipo de reemplazo hace que haya más diversidad de soluciones y que la población inicial no influya tanto. Esto lo podemos comprobar viendo los boxplots, donde las cajas de los AGG suelen ser las más compactas de todas, debido a que tienen menos variación en sus resultados; por tanto, la población inicial no influye tanto. En las tablas podemos ver que este tipo de algoritmos se ve beneficiado cuando el espacio de búsqueda es más grande, permitiéndole así obtener el mejor fitness promedio en las ejecuciones del SP100 y SP500.

6.4 Comportamiento de los Algoritmos Genéticos Estacionarios

Los AGE, al hacer reemplazos en proporciones pequeñas de la población, conservan mejor las soluciones que son buenas al inicio; esto los hace depender mucho de la población inicial. Podemos comprobar esto fijándonos en los boxplots, ya que el AGE es uno de los algoritmos que más outliers presenta y que más variabilidad tiene, sobre todo cuando se usa el operador aritmético por los motivos comentados anteriormente. Fijándonos en las tablas, vemos que en los dataset grandes claramente gana el AGG, pero en el ibex35 el AGE consigue superarlo debido a que en dataset pequeños la capacidad explotativa funciona mejor.

6.5 AGE-BLα vs AGG-BLα

En cuanto a comparación entre algoritmos genéticos nos fijaremos en los que usan el operador **BLX-α** para compararlos en igualdad de condiciones cuando usan el mejor operador.

Se puede observar fácilmente en los gráficos de líneas que ambos algoritmos tienen resultados muy parecidos; en datasets pequeños, como podría ser el del ibex35, vemos que no se aprecia apenas diferencia, ya que la ventaja explorativa del AGG no tiene apenas efecto. La diferencia se puede observar en el dataset de mayor tamaño, donde vemos que el AGG (línea morada) consigue mantener su fitness por encima del AGE en la mayoría de las ejecuciones. Por tanto, podemos ver que en datasets grandes se obtienen mejores soluciones cuando se tiene mayor diversidad en la población del algoritmo que cuando se intenta conservar de manera prematura a los mejores individuos (AGE).

6.6 Comparación entre meméticos

Los algoritmos meméticos usados son una hibridación entre el AGG-BLXα (el mejor de los genéticos) y la búsqueda local. Esto nos permite que cada vez que ejecutamos la búsqueda local sobre la cantidad de cromosomas seleccionados en cada caso, podamos explotar las soluciones que se encuentran en un entorno del cromosoma seleccionado. Como los resultados obtenidos en todas las variantes de los algoritmos son muy parecidos entre sí, podemos concluir dos cosas:

- O bien nuestro espacio de búsqueda no presenta «picos» sobre los que la búsqueda local pueda escalar para mejorar la solución inicial que se le da con el AGG.
- O bien debemos añadir más iteraciones a la búsqueda local para darle más posibilidades de poder encontrar mejores soluciones en el entorno dado.

Para comprobarlo, he probado a hacer ejecuciones usando 500 evaluaciones en la búsqueda local; como resultado he visto que la solución varía en cuestión de centésimas, por lo que considerar el aumento de evaluaciones en la búsqueda local no mejora los resultados.

Dentro de que los resultados entre meméticos no son muy distintos , podemos apreciar las diferencias entre AM-RAND y AM-BEST. El que mejor resultados obtiene es el AM-BEST ya que estamos aplicando una mejora (búsqueda local) sobre soluciones que ya son las mejores de la población por lo que tenemos más probabilidades de encontrar nuevas soluciones mejores.Por otro lado AM-RAND obtiene los peores resultados independientemente del dataset , esto se debe a que aplica la mejor solo sobre un 10% aleatorio de la población por lo que muchas veces se pueden estar mejorando soluciones las cuales son muy malas y no merece la pena mejorar ya que no nos aproximan a ninguna solución mejor.AM-ALL suele estar en un punto intermedio entre los dos anteriores , al aplicar la búsqueda local sobre todas las soluciones. Este enfoque tiene dos puntos negativos que no le permiten superar al AM-BEST:

- Gasta muchas evaluaciones haciendo la búsqueda local y deja menos iteraciones para el AGG
- El coste computacional es mucho más alto y esto lo podemos ver reflejado en el tiempo ya que es el algoritmo que más tarda en ejecutarse.

6.7 Mejor Memético vs mejor AG

En este último apartado voy a comparar el AGG-BLX-α con el AM-BEST. Observando los resultados de la tabla, vemos que el que mejor fitness obtiene es el AM-BEST. En el IBEX35 la diferencia es muy pequeña debido a las condiciones del dataset. Donde más diferencia podemos encontrar es en los otros dos casos. En el SP500 gana el AGG; esto puede deberse a que el algoritmo memético gasta evaluaciones en la búsqueda local intentando explotar las mejores soluciones de las que dispone, pero como ya hemos visto, en este dataset la exploración se ve mucho más recompensada que la explotación; por tanto, el AGG puede destinar más evaluaciones a explorar que el AM-BEST. El SP100 parece ser el entorno ideal para el AM-BEST, ya que es un dataset de tamaño intermedio donde vemos que sí puede aprovechar las evaluaciones que invierte en explotar las mejores soluciones. También vemos al mejor algoritmo en este dataset, y presenta muy poca variación en su boxplot, lo que nos indica que la población inicial no influye casi en su comportamiento.

6.8 Análisis resultados 2025

A la hora de analizar los resultados obtenidos en el año 2025. En cuanto al IBEX35, vemos que el mejor resultado lo obtiene el AGG-Arit, obteniendo un beneficio del 5,5%; como hemos visto en apartados anteriores, este algoritmo ha obtenido buenos resultados en los años previos, por lo que era de esperar que el beneficio fuese bueno. Los otros algoritmos también obtienen resultados aceptables, con una media del 3,2% de beneficio y con fitness muy parecidos entre ellos, por lo que vemos que las predicciones sobre 2025 son bastante buenas en este caso.

En el SP100, el beneficio obtenido baja drásticamente y todos los algoritmos se mueven en la zona del 1% de beneficio. No podemos sacar un claro ganador debido a que las diferencias son mínimas, pero el AGE-BLX junto al AM-BEST son los que mejores resultados obtienen. Tiene sentido, ya que han sido también de los que mejores resultados han obtenido en los datos de 2015-2024.

Por último, vemos que en el SP500 es donde peor funcionan las soluciones obtenidas. Los algoritmos AGG-Arit y AGE-Arit presentan pérdidas y los demás algoritmos tienen beneficios muy bajos o nulos. Esto nos dice que las soluciones obtenidas en este problema en el rango 2015-2024 no son buenas aplicadas al año 2025, ya que ninguna obtiene resultados buenos; es más, si nos fijamos en el Random search, vemos que este es el que mejor beneficio obtiene, por lo que las soluciones no están siendo adecuadas para este periodo.

Como resultado de este apartado, vemos que el algoritmo que mejor consigue extrapolar sus soluciones a 2025 es el AGG-BLX y AM-BEST, que aunque no son los que mejor resultado obtienen en todos los dataset, consiguen de media los mejores resultados y no generan pérdidas en ningún momento.

6.9 Conclusiones

Tras el análisis de los resultados, hemos podido comprobar que los algoritmos que mejores resultados han mostrado de forma consistente son AGG-BLX y AM-BEST. Esto se ha visto reflejado tanto en sus resultados durante 2014-2024 como en los resultados de 2025, donde han presentado los mejores resultados teniendo en cuenta todos los datasets.

Si analizamos el comportamiento general de los algoritmos sobre el problema, podemos concluir que nuestro espacio de búsqueda no presenta picos importantes donde los algoritmos meméticos hayan sacado mucha ventaja; el único que sí podría ser un poco más irregular sería el SP100, donde el AM-BEST consiguió superar a todos aprovechando sus cualidades.

Como conclusión, sacamos que en este problema se premia más la exploración que la explotación en el espacio de búsqueda, sobre todo en espacios muy grandes.